

计算机领域本科教育教学改革试点
工作计划（“101计划”）研究成果

人工智能引论

吴飞

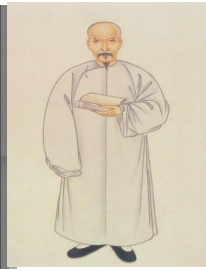
浙江大学计算机学院
浙江大学人工智能研究所

第三章

搜索探寻与问题求解

故记诵者，学问之舟车也

——清·章学诚《文史通义》



“学问之始，未能记诵；博涉既深，将趋记诵；故记诵者，学问之舟车也”。背诵是学问的基本功，犹如搜索算法从庞大解空间中搜索得到与问题相对应、满足一定约束条件的最佳答案，寻找一条从初始状态（问题所处状态）到目标状态（答案所处状态）的最佳路径。

提纲

一、搜索基本概念

二、贪婪最佳优先搜索与A*搜索

三、Minimax搜索与alpha-beta剪枝算法

四、蒙特卡洛树搜索

搜索的形式化描述

〈状态、动作、状态转移、路径/代价、目标测试〉

状态（state）

广义来说状态是对搜索算法和搜索环境当前所处情形的描述信息。为了简化求解过程，常常会剔除一些与问题求解无关细节描述（例如每个城市的人口情况等），只保留一些对求解模型产生作用的信息。在左图中，搜索算法要从城市A出发找到一条到城市K最短路线，影响其下一步行动的信息只有算法当前所位于的城市，因此每个城市即为一个状态。搜索算法刚开始所在状态（即城市A）被称为初始状态，完成任务时所在状态（即城市K）被称为终止状态。

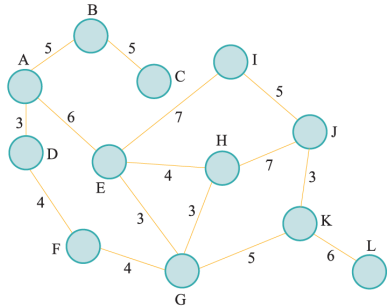


图 3.1　某地区的交通网络图

以求取从城市A到城市K之间一条行驶时间最短路线为例

搜索的形式化描述

〈状态、动作、状态转移、路径/代价、目标测试〉

动作（action）

广义来说状态是对搜索算法和搜索环境当前所处情形的描述信息。为了完成最短路径的探索，算法需要不断从一个状态转移到下一个状态。算法从一个状态转移到另外一个状态所采取的行为被称为动作。在这个问题中，动作即为乘坐一趟列车从当前城市到下一城市。显然在这个问题中，任何状态下可选择的动作都是离散的、而非连续，这个性质也适用于本课程讨论的所有问题。

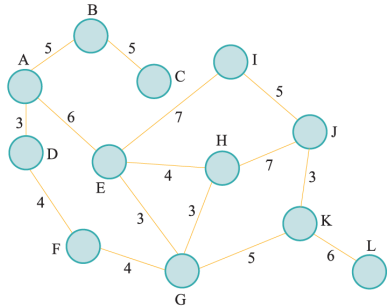


图 3.1　某地区的交通网络图

以求取从城市A到城市K之间一条行驶时间最短路线为例

搜索的形式化描述

〈状态、动作、状态转移、路径/代价、目标测试〉

状态转移（state transition）

算法选择了一个动作之后，其所处状态也会发生相应变化，这个过程被称为状态转移。如算法在当前城市A选择乘上开往邻接城市D的列车，那么将发生从状态A到状态D的状态转移。

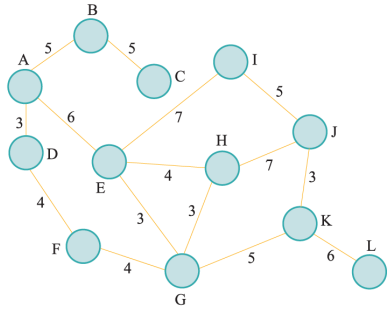


图 3.1　某地区的交通网络图

以求取从城市A到城市K之间一条行驶时间最短路线为例

搜索的形式化描述

〈状态、动作、状态转移、路径/代价、目标测试〉

路径（path）和代价（cost）

以任何一个状态为起点，搜索算法通过执行一系列动作后，将会在不同状态之间不断转移。将这个过程中经历的状态记录下来，可以得到一个状态序列，这个状态序列被称为一条路径。如从状态A出发，经过状态E、G，最终到达状态K，就形成了A → E → G → K这一路径。每条路径对应一个代价，图中路径的代价就是通过该路径的时间开销。如果一条路径对应的状态序列长度为2，那么这条路径的代价也被称为单步代价，图中每条边上的数值即为单步代价取值。一般假定单步代价取值为非负。

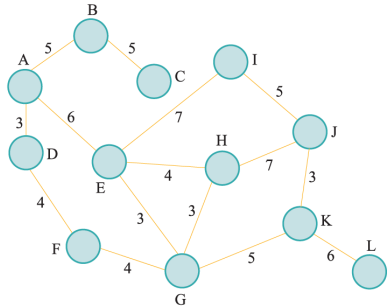


图 3.1　某地区的交通网络图

以求取从城市A到城市K之间一条行驶时间最短路线为例

搜索的形式化描述

〈状态、动作、状态转移、路径/代价、目标测试〉

目标测试（goal test）

目标测试函数 goal_test(s) 用于判断状态 s 是否为目标状态。在图中，如果当前状态为K，那么目标测试通过，否则不通过。目标测试通过意味着搜索算法完成，但是搜索算法完成并不意味着算法已经找到了最短路径。在最短路径搜索中，搜索算法会不断从当前结点的邻接节点中选择下一个结点，如此循环，直到找到一条从初始状态到终止状态的路径为止。当然，寻找所得的路径（从初始状态到终止状态）是否满足“路径最短”这一约束条件，还需要另行判断。

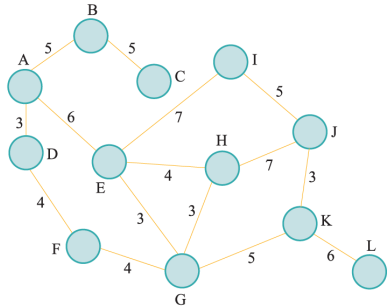


图 3.1　某地区的交通网络图

以求取从城市A到城市K之间一条行驶时间最短路线为例

搜索过程可视为搜索树的构建

- 可以用一棵树来记录算法所探索的路径，如左图给出了图3.1中部分搜索结果。这样一棵树被称为搜索树，树中每个结点用一个状态（城市）标记（即每个状态对应一个城市）。
- 图3.2中所示搜索树第三层中有三个标号均为A的结点，它们在图中分别被圆圈、正方形和三角形框住。虽然这三个标号A对应同一个城市，但是这三个结点在搜索树中却表示不同结点，因为它们分别代表了从初始状态出发到达城市A的如下三条不同路径：A → B → A、A → D → A和A → E → A。因此需要注意的是，在搜索树中，同一个标号一定表示相同的状态（即城市），其含义为搜索算法所在当前城市，但是同一个标号可能在搜索树中出现多次，不同的结点对应了从初始状态出发的不同路径。
- 可见，搜索树记录了从根结点出发目前所有探索过的路径。搜索算法可以被看成是一个构建搜索树的过程，从根结点（初始状态）开始，不断展开每个结点的后继结点，直到某个结点通过了目标测试。

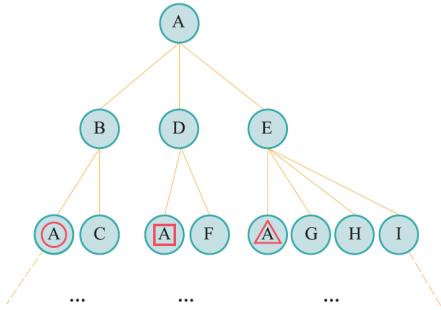


图 3.2　搜索树示例

（注：对应图 3.1 中的路径问题。搜索树第三层中标号为 *A* 的三个结点由于具有从初始结点到达的不同路径而属于不同结点）

搜索算法的评测标准

- **完备性**：当问题存在解时，算法是否能保证找到一个解，虽然这个解可能不是最优解。
- **最优性**：搜索算法是否能保证找到的第一个解是最优解。
- **时间复杂度**：找到一个搜索路径所需时间。
- **空间复杂度**：算法运行时所需的内存空间。

表 3.1　搜索树中用于估计复杂度的变量含义

符号	含义
b	分支因子，即搜索树中每个结点最大的分支数目
d	根结点到最浅的目标结点的路径长度
m	搜索树中路径的最大可能长度
n	状态空间中状态的数量

- 完备性和最优性刻画了算法找到解的能力以及所求的解的质量，时间复杂度和空间复杂度衡量了算法的资源消耗，通常用O符号（big O notation）来描述。
- 一般地，时间复杂度将通过扩展的结点数量、空间复杂度将通过算法同时记录的结点数量来衡量。
- 具体而言，搜索算法复杂度可能和以下变量有关：分支因子b，即搜索树中每个结点的最大分支数目；目标节点到根节点的最浅深度d，即搜索树中最早出现的目标结点所在层数；搜索树中路径的最大可能长度m；状态空间的大小n。

搜索框架

- 第1行对边缘集合进行初始化，将根结点作为边缘集合第一个元素
- 第3行从当前边缘集合中选出结点n
- 第4行将结点n从边缘集合中去除
- 第5-7行判断取出的结点n是否通过目标测试，如果通过目标测试，则返回从根结点到该结点的搜索路径。
- 第8行将结点n的后继结点加入边缘集合。
- 第3-8行的一系列操作通常被称为对结点n进行**扩展（expand）**。某个非目标结点被扩展意味着该结点从边缘集合$\mathcal{F}$中被取出（然后其被从该集合中删除），接着将该结点的后继结点放入边缘集合$\mathcal{F}$。

算法 3.1 简单的搜索算法框架

函数：TreeSearch

输入：结点选择函数 pick_from，后继结点计算函数 successor_nodes

输出：从初始状态到终止状态的路径

```
1  F ← { 根节点 }
2  while F ≠ ∅ do
3  n ← pick_from(F)
4  F ← F−{n}
5  if goal_test(n) then
6  | return n.path
7  end
8  F ← F ∪ successor_nodes(n)
9  end
```

- **函数pick_from：决定了扩展结点的顺序**
- **函数successor_nodes：决定了哪些结点可被放入边缘集合供后续被扩展**

搜索树构建示意

实际上，给定当前结点，并不是其所有的后继结点都值得被探索。在某些情况下，主动放弃一些后继结点能够提高搜索效率而不会影响最终搜索结果，甚至能解决无限循环（即算法不停机）问题。放弃扩展部分结点的做法被称为**剪枝（pruning）**，其对应的搜索算法被称为剪枝搜索。

图3.3中最左侧的路径为A → B → A → B → …，这意味着在某些搜索策略下（例如深度优先搜索），算法可能会沿着搜索树的左侧路径在状态A和状态B之间陷入无限循环，即出现环路或回路，搜索算法无法终止，这样算法将不具有完备性。

为了解决算法不完备的问题，一个直观办法是对可能产生环路的结点进行剪枝。举例来说，假设某时刻从根结点出发所构建得到的搜索树如图3.3所示。图中实线结点表示已经被扩展过，虚线结点为边缘集合中的结点（等待后续处理）。

对于图3.3中路径A → E → G而言，当结点G被扩展时，此时结点G被从边缘集合中取出并删除，结点G的后续结点{E, F, H, K}被放入边缘集合。对于所得的新边缘集合，第四层最左端的结点E不应该被拓展，否则会出现从E出发最后回到E的环路（E → G → E）。

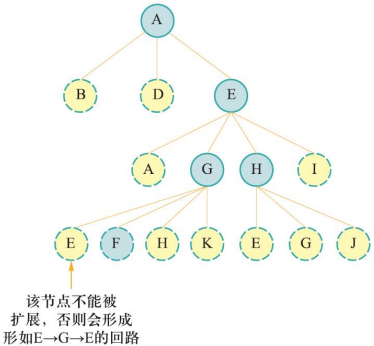


图 3.3 正在构建中的搜索树

（注：图中实线结点表示已被扩展，虚线结点在边缘集合中）

正在构建中的搜索树。图中实线结点表示已被扩展，虚线结点在边缘集合中。

贪婪最佳优先搜索：有信息（informed）搜索或启发式（heuristic）搜索

两个重要函数：评价函数与启发函数

辅助信息	所求解问题之外、与所求解问题相关的特定信息或知识。	
评价函数（evaluation function）f(n)	**从当前节点n出发，根据评价函数来选择后续结点**	**下一个结点是谁?**
启发函数（heuristic function）h(n)	**计算从结点n到目标结点之间所形成路径的最小代价值，这里将两点之间的直线距离作为启发函数**	**完成任务还需要多少代价?**

贪婪最佳优先搜索(Greedy best-first search)：评价函数f(n)=启发函数h(n)

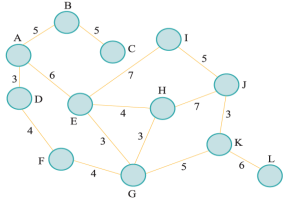


图 3.1　某地区的交通网络图

求取从城市A到城市K之间一条行驶时间最短路线

表 3.2　每个城市到目标城市的直线距离（启发函数取值）

状态	A	B	C	D	E	F	G	H	I	J	K	L
$h(n)$	13	10	6	12	7	8	5	3	6	3	0	6

每个城市到目标城市（即城市K）的直线距离（启发函数取值）

贪婪最佳优先搜索：将启发函数作为评价函数的搜索过程

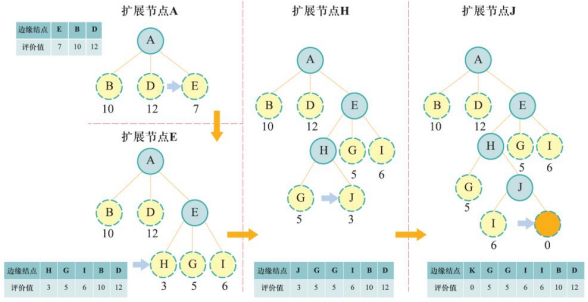


图 3.4　贪婪最佳优先搜索的过程

（注：图中边缘结点下的数字表示其评价函数取值）

贪婪最佳优先搜索的过程。
图中边缘结点下数字表示其评价函数取值

- 图中仍然用实线结点表示已经被扩展过的结点，虚线结点表示边缘结点。
- 边缘结点下的数字表示该结点到目标结点的直线距离，这一距离取值被用来评价当前结点的后续结点选择。需要提醒的是，这一距离取值既是启发函数取值，也是评价函数取值。

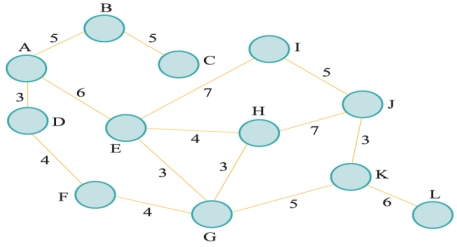


图 3.1　某地区的交通网络图

求取从城市A到城市K之间
一条行驶时间最短路线

表 3.2　每个城市到目标城市的直线距离（启发函数取值）

状态	A	B	C	D	E	F	G	H	I	J	K	L
$h(n)$	13	10	6	12	7	8	5	3	6	3	0	6

每个城市到目标城市（即城市K）的直线距离（启发函数取值）

贪婪最佳优先搜索：将启发函数作为评价函数的搜索过程

算法找到了一条从起始结点到目标结点的路径A → E → H → J → K
但这条路径并不是最短路径，实际上最短路径为A → E → G → K

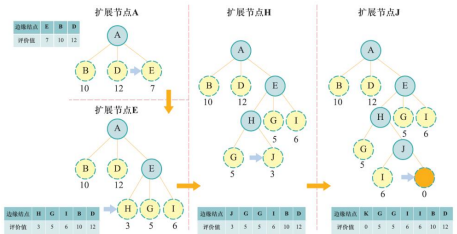


图 3.4　贪婪最佳优先搜索的过程

（注：图中边缘结点下的数字表示其评价函数取值）

贪婪最佳优先搜索的过程

A → E → H → J → K （✗）

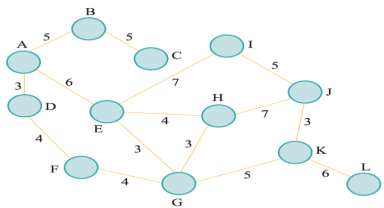


图 3.1　某地区的交通网络图

求取从城市A到城市K之间一条行驶时间最短路线

A → E → G → K (✔)

真正的最短路线

贪婪最佳优先搜索：为什么会失败

欲速则不达

- 贪婪最佳优先搜索算法采用了排除环路的剪枝方法，因此它是完备的，即总是可以找到一个解。但是该算法并不是最优的。
- 在最坏的情况下，贪婪最佳优先搜索算法时间和空间复杂度均为$O(b^m)$，其中b和m分别表示搜索树的分支因子和最大深度。
- 贪婪最佳优先搜索算法的问题在于其总是贪婪地扩展与目标结点之间代价最小（直线距离最近）的结点，而不考虑从初始结点到达该结点所需的代价。
- 在上述搜索步骤中，虽然第四步扩展的J结点距离目标结点的估计代价很小（取值为3），但从起始结点到达该结点的路径A → E → H → J代价为17，已经超过了实际上最短路径的代价（值为14），因此路径A → E → H → J并不是一个好的探索方向。

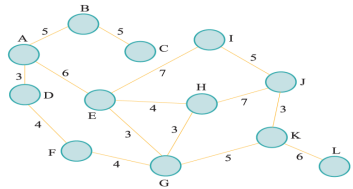


图 3.1　某地区的交通网络图

求取从城市A到城市K之间一条行驶时间最短路线

表 3.2　每个城市到目标城市的直线距离（启发函数取值）

状态	A	B	C	D	E	F	G	H	I	J	K	L
$h(n)$	13	10	6	12	7	8	5	3	6	3	0	6

每个城市到目标城市（即城市K）的直线距离（启发函数取值）

A*搜索算法：评价函数和启发函数各司其职

在评价函数中考虑从起始结点到当前结点的路径代价

$$\underbrace{f(n)}_{\text{评价函数}} = \underbrace{g(n)}_{\text{起始结点到结点}n\text{代价}} + \underbrace{h(n)}_{\text{结点}n\text{到目标结点代价}}$$

$$\underbrace{\quad}_{\text{(选择后续结点)}} \qquad \underbrace{\quad}_{\text{(当前最小代价)}} \qquad \underbrace{\quad}_{\text{(后续估计最小代价)}}$$

辅助信息	所求解问题之外、与所求解问题相关的特定信息或知识。	
评价函数（evaluation function）f(n)	**从当前节点n出发，根据评价函数来选择后续结点。**	**下一个结点是谁?**
启发函数（heuristic function）h(n)	**计算从结点n到目标结点之间所形成路径的最小代价值，这里将两点之间的直线距离作为启发函数。**	**完成任务还需要多少代价?**

定义从起始结点到结点n的路径代价函数$g(n)$，在此基础上评价函数$f(n)$被修改为：$f(n)=g(n)+h(n)$，即评价函数等于从起始结点到达结点n的的实际代价取值与结点n到目标结点估计代价取值之和。根据每个结点的评价函数之值不断选择后续结点的搜索算法被称为A*算法

A*搜索算法搜索过程

- 图3.5中用实线结点表示已经被扩展过的结点，虚线结点表示边缘结点。每个边缘结点下方等式即为$g(n)+h(n)=f(n)$的取值。
- 首先扩展根结点（起始结点）A，得到三个边缘结点B、D和E。其中路径A → B的实际代价为5，结点B的启发函数取值为10，因此结点B的评价函数取值为$5+10=15$。同理可算出D和E的评价函数值分别为15和13。算法选择评价函数值最小的E结点进行扩展，得到评价函数取值分别为13、14、19的三个新的边缘结点H、G、I。算法从当前边缘集合中选择评价函数值最小的结点H进行扩展，得到评价函数值为18和20的两个结点。到目前为止，算法的流程和贪婪最佳优先搜索保持一致，两者区别体现在下一步操作上。
- 不同于贪婪最佳优先搜索选择扩展结点J，A*算法选择扩展评价函数值最小的第三层中结点G（A → E → G），接着又扩展了评价函数值最小的结点K，顺利到达终止状态。此处扩展次序的差异使得A*算法能够顺利找到最短路径A → E → G → K，而贪婪最佳优先搜索则不能。
- 值得注意的是，每次从边缘结点集合中所扩展的结点并非总是当前结点的相邻后继结点，如在图中，所扩展的第三层结点G（A → E → G）并不是先前扩展结点H(A → E → H）在搜索树上的相邻后继结点。

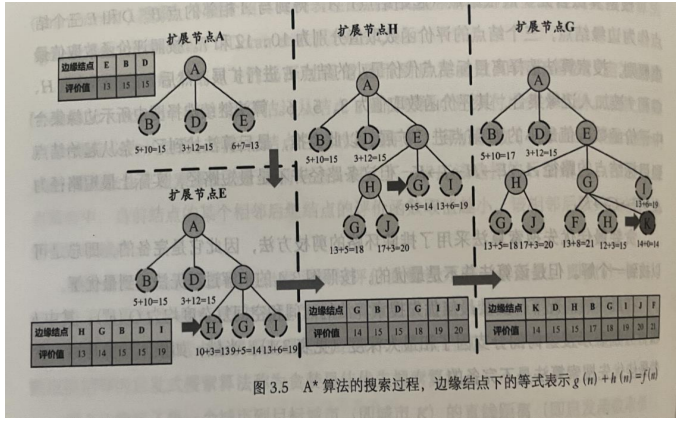


图 3.5　A* 算法的搜索过程，边缘结点下的等式表示 $g(n)+h(n)=f(n)$

A*搜索性能分析

A*算法的完备性和最优性取决于搜索问题和启发函数的性质。

- 可容性（admissible）
- 一致性（consistency）

表 3.3　本节中使用的若干符号

符号	含义
$h(n)$	结点 n 的启发函数取值
$g(n)$	从起始结点到结点 n 所对应路径的代价
$f(n)$	结点 n 的评价函数取值
$c(n, a, n')$	从结点 n 执行动作 a 到达结点 n' 的单步代价
$h^*(n)$	从结点 n 出发到达终止结点的最小代价

A*搜索性能分析：可容性

对于任意结点n，有$h(n) \leq h^*(n)$，如果n是目标结点，则有$h(n)=0$。$h^*(n)$是从结点n出发到达终止结点所付出的最小代价。可以这样理解满足可容性的启发函数，启发函数不会过高估计（over-estimate）从结点n到终止结点所应该付出的代价（即估计代价小于等于实际代价）。在上述问题中，对每个结点而言，其状态（城市）到目标状态（城市）K之间的行驶距离不会小于两城之间的直接距离，因此列车以最大行驶速度通过直线距离花费的时间必然不会超过实际代价，将该时间作为启发函数值，是满足可容性条件的。

A*搜索性能分析：一致性

启发函数的一致性指满足条件 $h(n) \leq c(n,a,n') + h(n')$，这里$c(n,a,n')$表示结点$n$通过动作$a$到达其相应的后继结点$n'$的代价。在图3.1.1中，假设结点$n$和$n'$对应的状态（城市）为$s$和$s'$，如果将单步代价$c(n,a,n')$重新定义为$s$和$s'$在二维平面上的欧氏距离，则根据欧氏距离三角不等式原则，城市s到目标城市K之间直线距离一定小于等于从城市s到其相邻城市s'的直线距离与城市s'到目标城市K之间直线距离之和，此时，将当前状态（城市）与目标状态（城市）K之间直线距离作为启发函数值，则启发函数满足一致性条件。

A*搜索性能分析：一致性必然导致可容性

- 一致性是一个比可容性更严格的性质，即一个启发函数如果满足一致性条件，那么该启发函数必然满足可容性条件。

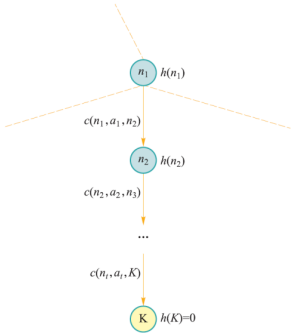


图 3.6　从 n_1 结点到终止结点的最短路径

如图中以n_1为根结点的子树，假设从该结点到达终止结点代价最小的路径为$n_1 \to n_2 \to \cdots \to$ K。由于$h(\mathrm{K}) = 0$且启发函数满足一致性条件，因此存在：

$$\begin{aligned} h(n_1) &\le h(n_2) + c(n_1, a_1, n_2) \\ &\le h(n_3) + c(n_2, a_2, n_3) + c(n_1, a_1, n_2) \\ &\le \cdots \\ &\le c(n_1, a_1, n_2) + c(n_2, a_2, n_3) + \cdots \\ &= h^*(n_1) \end{aligned}$$

即满足一致性条件的启发函数一定满足可容性条件。

A*搜索性能分析：A*算法是完备的么?

如果所求解问题和启发函数满足以下条件，则A*算法是完备的:

- 搜索树中分支数量是有限的，即每个结点的后继结点数量是有限的
- 单步代价的下界是一个正数
- 启发函数有下界

A*搜索性能分析：A*算法是最优的么？

树搜索和图搜索的区别
A*算法

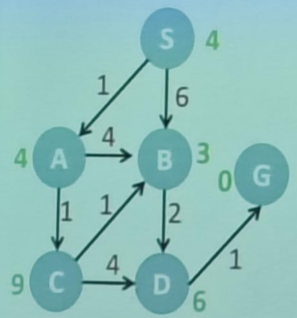
如果启发函数是可容的，那么A*算法满足最优性

假设A*算法找到的第一个终止结点为n。对于此时边缘集合中任意结点n'，根据算法每次扩展时的策略，即选择评价函数取值最小的边缘结点进行扩展，有$f(n) \leq f(n')$。由于A*算法对评价函数定义为$f(n) = g(n) + h(n)$，且$h(n) = 0$，有$f(n) = g(n) + h(n) = g(n)$，$f(n') = g(n') + h(n')$，综合可得

$$g(n) \leq g(n') + h(n') \leq g(n') + h^*(n')$$

其中$g(n)$为从起始结点到结点n所对应路径的代价，$g(n') + h^*(n')$表示从初始结点出发经过结点n'到达终止结点的最小代价。也就是说此时扩展其他任何一个边缘结点都不可能找到比结点n所对应路径代价更小的路径，因此结点n对应的是一条最短路径，即算法满足最优性。

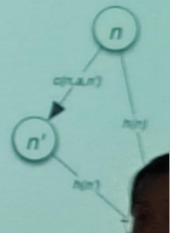
Example: A* graph search



Expnd. node	Open list	Closed list
	{S：4}	
S	{A:5, B:9}	{S}
A	{B:8, C:11}	{S,A}
B	{C:11, D:13}	{S,A,B}
C	{B:6, D:12}	{S,A,C}
B	{D:11}	{S,A,C,B}
D	{G:6}	{S,A,C,B,D}

不满足 $h(n) - h(n') \le c(n,a,n')$ 的有？

SA：1+AC：1+CB：1+BD：2+DG：1=6



- 在通常情况下，搜索问题的状态数量是有限的，因此图搜索的A*算法和排除环路的树搜索 A*算法均是完备的，均有解。

- 思考：

（1）那么使得 A*算法是完备的，需要所求解问题及启发函数满足哪些条件？

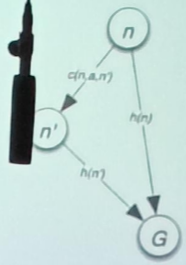
（2）为什么基于树搜索的 A*算法，如果启发函数是可容的，则算法一定满足最优性，而基于图搜索的 A*算法却无法得出同样的结论？

上传网络！

改进的A*算法(Graphic Search)

Algorithm A*

OPEN={S}
while OPEN is not empty
{ 将OPEN中$f(n)$最小的节点n移入CLOSED
if n is a goal node,
return success (path p)
对n 的所有子节点 m
if m is on CLOSED 且当前代价比原来的代价低,
then remove m from CLOSED,
put m on OPEN
else if m is on OPEN且当前代价比原来的代价低,
修改原OPEN表中m的父节点指针
else if m is not on OPEN
Then put m on OPEN }
Return failure



条件：$h(n) - h(n') \le c(n,a,n')$

若启发函数满足单调性（一致性，consistent），则对应的A*算法称为改进的A*算法。

Minimax搜索：对抗搜索（adversarial search）或博弈搜索（game search）

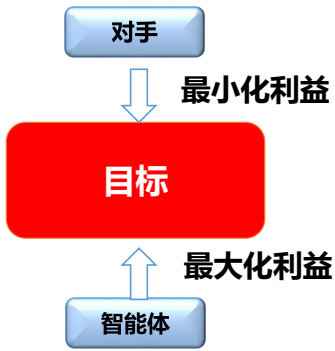
两个智能体同处于一个竞争的环境，智能体之间通过竞争来实现各自相反目标，即一方想**最大化自身的利益**，而另一方则想**最小化对手的利益**。通俗地说，对抗搜索的过程就是两个智能体各自选择对自己有利的策略。

狭路相逢勇者胜

勇者相逢智者胜

智者相逢德者胜

德者相逢道者胜



对抗搜索跟盲目搜索

启发式搜索有什么不同?

零和博弈：田忌赛马，井字棋

Minimax搜索：若干定义

- **状态**：状态s包括当前的游戏局面和当前行动的智能体。初始状态s_0包括初始游戏局面和初始行动的玩家。由于本节讨论的问题假设两个竞争对手轮流行动，因此第i步行动的玩家是确定的，函数player(s)给出状态s下行动的智能体。
- **动作**：给定状态s，动作指的是player(s)在当前局面下可以采取的操作a，记动作集合为actions(s)。
- **状态转移**：给定状态s和动作$a \in$ actions(s)，状态转移函数result(s,a)决定了在s状态采取a动作后所得后继状态。
- **终局状态检测**：终止状态检测函数terminal_test(s)用于测试游戏是否在状态s结束。
- **终局得分**：终局得分utility(s,p)表示在终局状态s时玩家p的得分。在二人零和博弈中，两名玩家的终局得分之和应该是固定的，因此算法只需记录其中一人的终局得分为utility(s)，则另一人的得分可按照零和原则相应算出。

Minimax搜索：搜索策略

由于每个玩家都想要最大化自己的得分，所以轮到玩家1行动时，他会总是选择能够最大化终局得分的搜索树分支，而玩家0总是选择能够最小化终局得分的搜索树分支，最小最大算法的名字即由此而来

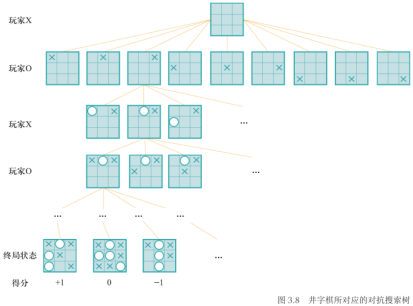


图 3.8　井字棋所对应的对抗搜索树

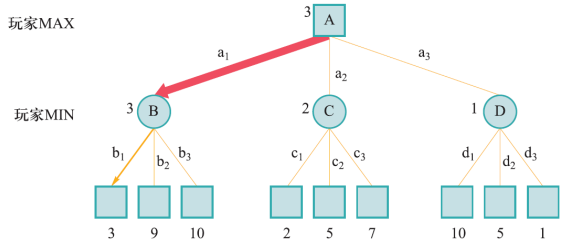


图 3.9　玩家 MAX 和玩家 MIN 在最优策略下采取动作的示意

$$\text{minimax}(s) = \begin{cases} \text{utility}(s), & \text{if terminal_test}(s) \\ \max_{a \in \text{actions}(s)} \text{minimax}(\text{result}(s,a)), & \text{if player}(s) = MAX \\ \min_{a \in \text{actions}(s)} \text{minimax}(\text{result}(s,a)), & \text{if player}(s) = MIN \end{cases}$$

Minimax搜索：搜索算法流程

Minimax搜索就是在计算搜索树中每个结点分数之后，每一步由玩家根据自己的角色来选择使得分数最大或分数最小的行动。

算法3.2给出了Minimax算法的流程，该算法假设玩家MAX为初始行动玩家。这个假设并不会使问题失去一般性，如果实际上初始行动的是玩家MIN，则可以通过将终局得分取相反数的方法来交换玩家MIN和玩家MAX。函数MinValue和MaxValue递归求解了每个搜索树结点的分数，函数MinimaxDecision根据每个结点的分数求解玩家MAX在当前盘面下所采取的最优动作。

Minimax算法对游戏搜索树执行了一个完整的深度优先搜索。因此，如果搜索树是有限的，那么Minimax算法必然能在有限时间内终止；如果对手总是理性地按照最优策略来交替采取行动，那么Minimax算法得到的解是最优的。如果m是游戏树的最大深度，在每个结点最多存在b种有效走法，那么Minimax算法的时间复杂度为$O(b^m)$，空间复杂度为$O(bm)$。

算法 3.2 通过最小最大搜索求解最优动作的算法

函数：MinimaxDecision
输入：当前的盘面状态 s
输出：玩家 MAX 行动下，当前最优动作 a^*

$a^* \leftarrow \operatorname{argmax}_{a \in \text{actions}(s)} \text{MinValue}(\text{result}(s, a))$

函数：MaxValue
输入：当前的盘面状态 s
输出：玩家 MAX 行动下，当前状态的得分 $v = \text{minimax}(s, \text{MAX})$

```
if terminal_test(s) then return utility(s)
v ← −∞
foreach a ∈ actions(s) do
    v ← max(v, MinValue(result(s, a)))
end
```

函数：MinValue
输入：当前的盘面状态 s
输出：玩家 MIN 行动下，当前状态的得分 $v = \text{minimax}(s, \text{MIN})$

```
if terminal_test(s) then return utility(s)
v ← +∞
foreach a ∈ actions(s) do
    v ← min(v, MaxValue(result(s, a)))
end
```

alpha-beta剪枝算法

Alpha-Beta 剪枝搜索算法在Minimax算法中可减少被搜索的结点数，即在保证得到与原Minimax算法同样的搜索结果时，剪去了不影响最终结果的搜索分枝。

$$\text{minimax}(A) = \max(\min(3,9,10), \min(2,x,y), \min(10,5,1))$$

由于$\min(2,x,y)$的值小于等于2，所以即使不知道x和y的值，根结点的分数也可以算出来等于3。也就是说，算法没有必要计算动作c_2和c_3对应的两个子树在游戏结束时所得分数，就能决定在根结点采取动作a_1从而在游戏结束时可获得收益3，因此动作c_2和c_3所对应子树可以被剪枝。

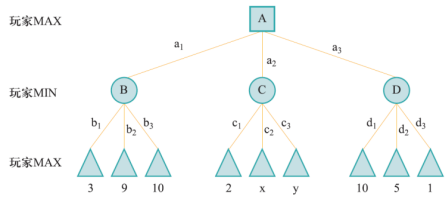


图 3.10　一棵对抗搜索树在搜索过程中某时刻的状态示意

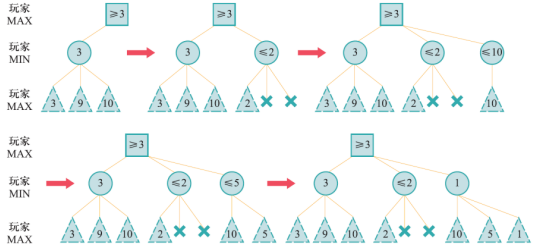


图 3.11　存在剪枝的搜索树部分扩展过程（对应图 3.10 的搜索树）

alpha-beta剪枝算法：对min节点后代剪枝示意图

- 可把上述剪枝思路推广到一般情况。假设有一个位于MIN层的结点m，已知该结点能够向其上MAX结点反馈的收益为α（alpha）。n是与结点m位于同一层的某个兄弟（sibling）结点的后代结点。如果在结点n的后代结点被访问一部分后，知道结点n能够向其上一层MAX结点反馈收益小于α，则结点n的未被访问孩子结点将被剪枝。

- 如图3.12所示，玩家MAX在结点m'能够从位于其下MIN层的结点m得到取值至少为α的收益，因此结点n那些未被访问的后继分支结点对结点m'取值没有任何影响了。

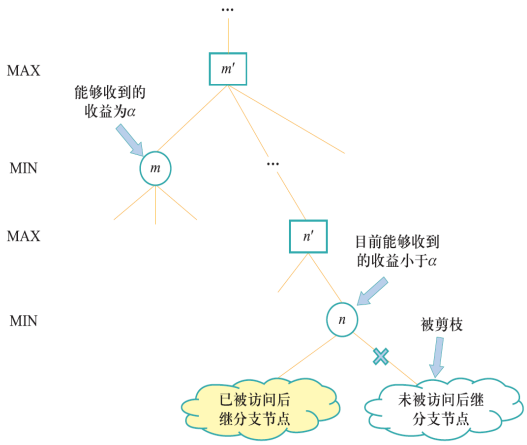


图 3.12　基于 MIN 结点的反馈收益进行剪枝（alpha 剪枝）

alpha-beta剪枝算法：对max节点后代剪枝示意图

- 如图3.13所示，考虑位于MAX层的结点m，已知结点m能够从其下MIN层结点收到的收益为β（beta）。
- 结点n是结点m上层结点m'的位于MAX层的后代结点，如果目前已知结点n能够收到的收益大于β，则不再扩展结点n的未被访问后继结点，因为位于MIN层的结点m'只会选择收益小于或等于β的结点来采取行动。

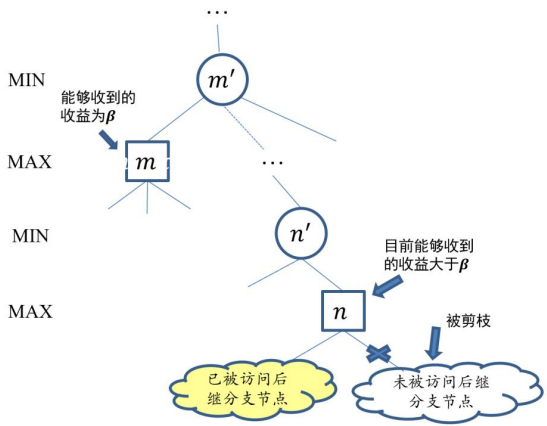
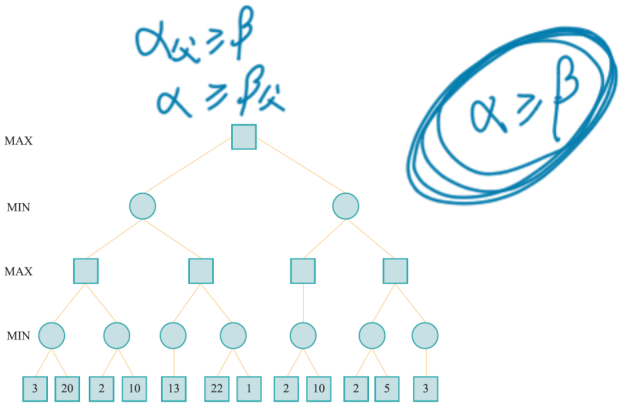


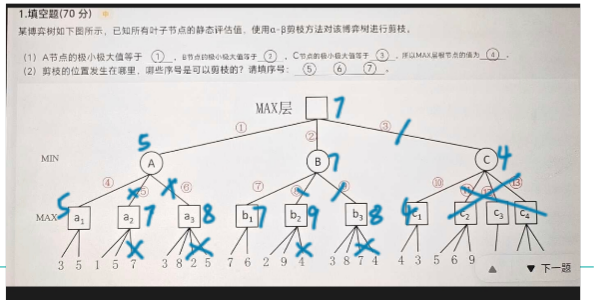
图 3.13 基于MAX结点反馈收益进行剪枝(beta剪枝)

alpha-beta剪枝算法：为什么叫alpha和beta剪枝

- 综合以上两种情况的剪枝方法被称为alpha-beta剪枝。实际上，在图3.12中，结点m不仅为MIN层结点n提供了一个下界，对于例如n'的MAX层结点，这个下界仍然是有意义的。图3.13中情况同理。因此，无论一个结点位于MIN层还是MAX层，都可以根据图3.12和图3.13中的情况找到一个下界和一个上界，即可以为每个结点设置一个α值和一个β值，来判断该结点及其后继结点是否可被剪枝。在设计算法时，为了计算一个结点的上下界，没有必要去枚举所有会影响当前结点α值和β值的结点，只需要继承父结点的α值和β值，再按照一定的规则加以更新即可。对于MAX结点，如果其孩子结点（MIN结点）的收益大于当前的α值，则将α值更新为该收益；对于MIN结点，如果其孩子结点（MAX结点）的收益小于当前的β值，则将β值更新为该收益。根结点（MAX结点）的α值和β值分别被初始化为$-\infty$和$+\infty$。
- 随着搜索算法不断被执行，每个结点的α值和β值不断被更新。大体来说，每个结点的$[\alpha,\beta]$从其父结点提供的初始值开始，取值按照如下形式变化：α逐渐增加、β逐渐减少。不难验证，如果一个结点的α值和β值满足$\alpha > \beta$的条件，则该结点尚未被访问的后续结点就会被剪枝，因而不会被智能体访问。

alpha-beta剪枝算法示意图





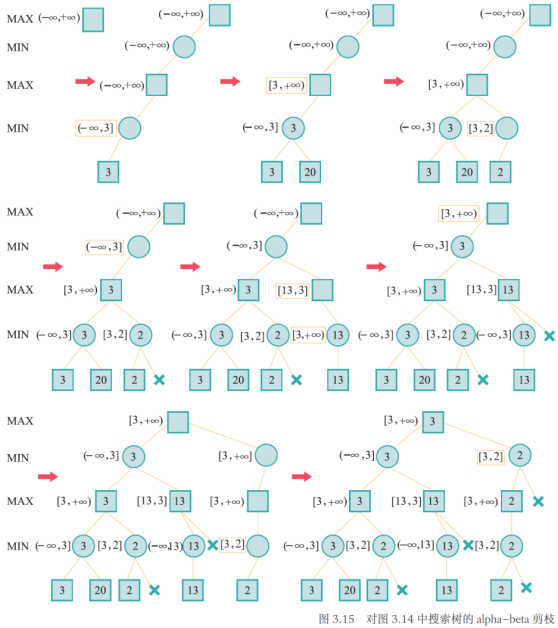


图 3.15　对图 3.14 中搜索树的 alpha−beta 剪枝

蒙特卡洛树搜索

以经典的多臂赌博机（Multi-armed Bandit）问题为例。先考虑一个简化问题：假设智能体面前有K个赌博机，每个赌博机有一个臂膀。每次转动一个赌博机臂膀，赌博机则会随机吐出一些硬币或不吐出硬币，将每次吐出的硬币的币值表示为收益分数。现在假设给智能体$\tau(\tau > K)$次转动臂膀的机会，那么智能体如何选择赌博机、转动τ次赌博机臂膀，以获得更多的收益分数呢？

或许可以让智能体先把K个赌博机的臂膀依次转动一遍，观察在摇动每个赌博机臂膀时的收益分数，然后去转动那些收益分数高的赌博机臂膀。但是，由于从每个赌博机获得的收益分数是随机的，一个刚刚给用户带来可观收益分数的赌博机在下一次摇动其臂膀时可能不会继续获得可观收益分数，因此这一方法不可取。那么是否有方法能够指导智能体的高效地获取收益分数呢？

蒙特卡洛树搜索：要素

- **状态**：每个被摇动的臂膀即为一个状态，记K个状态分别为$\{s_1, s_2, \cdots, s_K\}$，没有摇动任何臂膀的初始状态记为$s_0$。
- **动作**：动作对应着摇动一个赌博机的臂膀，在多臂赌博机问题中，任意状态下的动作集合都为$\{a_1, a_2 \cdots, a_K\}$，分别对应摇动某个赌博机的臂膀。
- **状态转移**：选择动作$a_i(1 \leq i \leq K)$后，将状态相应地转换为s_i。然而这个问题和前两节讨论的搜索问题的不同之处在于，由于存在随机性，智能体摇动赌博机臂膀若干个τ次，任意两个τ次的结果可能都不一样，为此引入如下奖励要素。
- **奖励（reward）**：假设从第i个赌博机获得收益分数的分布为D_i，其均值为μ_i。如果智能体在第t次行动中选择转动了第l_t个赌博机臂膀，那么智能体在第t次行动中所得收益分数$\hat{r}_t$服从分布D_{l_t}，$\hat{r}_t$被称为第t次行动的奖励。为了方便对多臂赌博机问题的理论研究，一般假定奖励是有界的，进一步可假设奖励的取值范围为$[0,1]$。

蒙特卡洛树搜索：要素

◆ **悔值（regret）函数**

根据智能体前T次动作，可以如下定义悔值函数：

$$\rho_T = T\mu^* - \sum_{t=1}^{T} \hat{r}_t$$

其中$\mu^* = \max_{i=1,\cdots,K} \mu_i$。显然为了尽量减少悔恨，在每次操作时，智能体应该总是转动能够提供最大期望奖励的赌博机臂膀，但是这是不现实的，因为智能体并不知道哪个臂膀的奖励期望最大。这一公式告诉我们，将T次操作中最优策略的期望得分减去智能体的实际得分，就是悔值函数的结果。显然，问题求解的目标为最小化悔值函数的期望，该悔值函数的取值取决于智能体所采取的策略。

蒙特卡洛树搜索：策略计算

智能体不能预先计算好最优策略，然后实行，而是需要一边探索一边调整自己的策略，争取获得更好的收益。

一种直观的做法是，智能体记录下每次摇动的赌博机臂膀和获得的相应收益分数。给定第$i(1 \le i \le K)$个赌博机，记在过去$t-1$次摇动赌博机臂膀的行动中，一共摇动第i个赌博机臂膀的次数为第$T_{(i,t-1)}$。于是，可以计算得到第i个赌博机在过去$T_{(i,t-1)}$次被摇动过程中的收益分数平均值$\bar{x}_{i,T_{(i,t-1)}}$。这样，智能体在第t步，只要选择$\bar{x}_{i,T_{(i,t-1)}}$值最大的赌博机臂膀进行摇动，这是贪心算法的思路。

假设一共有五台赌博机（即$K=5$），第i个赌博机的得分满足均值为μ_i、方差为1/75（即支撑区间长度为0.4）的均匀分布，其中均值$(\mu_1,\cdots,\mu_5)=(0.3,0.4,0.5,0.6,0.7)$。假设某个赌博机的臂膀没有被摇动时，因为无法计算其平均收益分数，不妨将这个赌博机得分均值设定正无穷大。在计算机上模拟执行$T=10000$次贪心算法，来摇动五台赌博机。可见在10000次模拟中，智能体趋向于摇动4号赌博机，其他编号赌博机被摇动次数很少。由于其他编号赌博机被摇动次数较少，可见对这些赌博机平均收益分数的估计结果与实际均值结果相差很大。

需要说明的是，如果重新进行一个10000次贪心算法的模拟，由于随机性的存在，其结果会存在不同。比如在另外一个10000次模拟中，贪心算法首先选择编号为2的赌博机臂膀摇动多次，则会导致后续行动中会对编号为2的赌博机摇动次数最多，忽略了其他编号的赌博机。

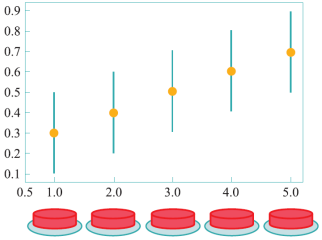


图 3.16　多臂赌博机问题中每个赌博机收益分数的分布情况

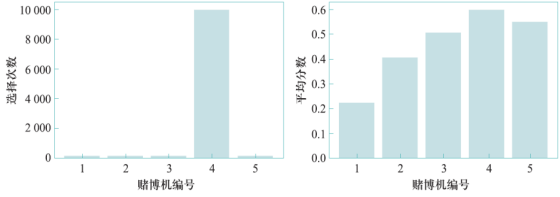


图 3.17　贪心算法选取 10 000 个动作的一次模拟实验结果

蒙特卡洛树搜索：盲从过去历史而导致不足

在上面的实验中，可以发现：贪心算法对编号4的赌博机“情有独钟”，很少去摇动其他编号的赌博机。因此，虽然对编号4赌博机的平均收益分数估计越来越准确，但是却难以尝试去摇动其他编号赌博机的臂膀，从而无法对其他赌博机的平均收益分数就行准确估计。而这些不准确的分数估计又反过来让智能体错误地认为5号赌博机不如4号赌博机，因而无法做出更好的选择。

上述困境体现了探索（exploration）和利用（exploitation）之间存在对立关系。贪心算法基本上是**利用**从已有尝试结果中所得估计来指导后续动作（如倾向于选择4号赌博机），但问题是所得估计往往不能准确反映未被（大量）探索过的动作，如由于很少摇动其他编号的赌博机而无法准确估计其他编号赌博机可能带来的收益分数。因此，需要在贪心算法中增加一个能够改变其“惯性”的内在动力，以使得贪心算法能够访问那些尚未被（充分）访问过的空间。

蒙特卡洛树搜索：ϵ-贪心算法

ϵ-贪心算法就是这样一种在探索与利用之间进行平衡的搜索算法。在第t步，ϵ-贪心算法按照如下机制来选择摇动赌博机：

$$l_t = \begin{cases} \text{argmax}_i \, \bar{x}_{i,T(i,t-1)}, & \text{以} 1-\epsilon \text{的概率} \\ \text{随机的} i \in \{1,2,\cdots,K\}, & \text{以} \epsilon \text{的概率} \end{cases}$$

即以$1-\epsilon$的概率选择在过去$t-1$次摇动赌博机臂膀行动中所得平均收益分数最高的赌博机进行摇动；以ϵ的概率随机选择一个赌博机进行摇动。

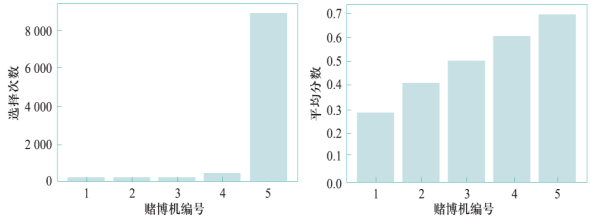


图 3.18　在图 3.17 的基础上再进行 10 000 次 $\epsilon-$贪心算法的实验结果

ϵ-贪心算法对估计摇动五个赌博机臂膀所带来期望分数更加准确了。并且因为编号为5的赌博机的平均收益分数最高，算法倾向于选择动作a_5，与预期相符。这说明在ϵ-贪心算法中加入探索机制发挥了作用。但是，如果继续使用贪心算法而不引入探索机制，那么算法极有可能继续选择10000个a_4动作、而继续忽略其他编号的赌博机。

蒙特卡洛树搜索：上限置信区间算法

ϵ-贪心算法虽然能有效地促使算法进行探索，但通过这种随机机制来选择动作进行探索的做法很可能不是最优的。直观来看，执行一个动作后所得奖励的样本少，那么算法对这个动作的奖励期望估计会不准确。比如，可能存在一个给出更好奖励期望的动作，但因为智能体对其探索次数少而认为其期望奖励小。因此，需要对那些探索次数少或几乎没有被探索过的动作赋予更高的优先级。

但是ϵ-贪心算法没有将每个动作被探索的次数纳入考虑，这似乎是不合理的。另一方面，简单地优先去尝试探索次数少的动作也未必合理，因为只需要少量的探索次数，就能够较好估计方差较小的动作的奖励期望。

总结以上想法，在探索过程中，应该优先探索估计值不确定度高动作。另外还应注意到，如果从已有探索样本中知道一个动作的奖励估计值极端偏小，那么即使该动作的估计值不确定度很大，算法也没有必要将其作为优先探索对象。上限置信区间（Upper Confidence Bounds, UCB1）算法正是采用这一思路来进行探索。

蒙特卡洛树搜索：上限置信区间算法策略

UCB1算法的策略是：为每个动作的奖励期望计算一个估计范围，优先采用估计范围上限较高的动作。如图3.19所示，动作1的奖励期望取值的不确定度（估计范围）虽然最大，但是因为其均值太小，因此UCB1算法不优先考虑探索动作1。动作2和3的奖励期望的均值相同，但是动作2的奖励期望取值的不确定度（估计范围）更大，于是因为置信上限更大，动作2会被UCB1算法优先考虑。

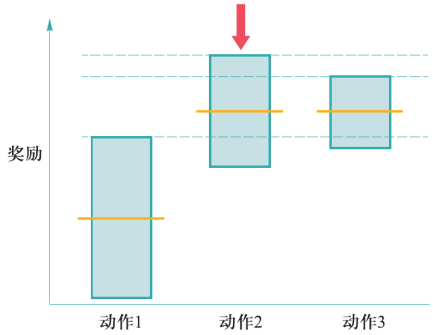


图 3.19　UCB1 算法的策略示意

蒙特卡洛树搜索：上限置信区间算法策略

接下来的问题在于算法应该如何计算奖励期望的估计范围，尤其是奖励期望的置信上界呢？假设算法在过去第t次已经对动作a_i探索了$T_{(i,t-1)}$次，在当前问题中对应摇动了第i个赌博机的臂膀$T_{(i,t-1)}$次，执行动作a_i所收到收益分数的均值为$\bar{x}_{i,T_{(i,t-1)}}$。这$T_{(i,t-1)}$次动作可以看作$T_{(i,t-1)}$个取值范围在$[0,1]$的独立同分布随机变量的样本，根据霍夫丁不等式（Hoeffding's inequality），有如下公式存在：

$$P\left(\mu_i - \bar{x}_{i,T_{(i,t-1)}} > \delta\right) \leq e^{-2T_{(i,t-1)}\delta^2}$$

这个不等式的含义是第i个动作的奖励期望μ_i大于$\bar{x}_{i,T_{(i,t-1)}} + \delta$之和的概率不超过$e^{-2T_{(i,t-1)}\delta^2}$，其中$\delta > 0$。因此只需找到一个$\delta$，使得上述不等式右侧足够小，即可认为$\bar{x}_{i,T_{(i,t-1)}} + \delta$是$\mu_i$的一个上界。

这里可找到一个随时间增长快速趋近于0的函数t^{-4}，令$e^{-2T_{(i,t-1)}\delta^2} = t^{-4}$，则$\delta = \sqrt{\frac{2\ln t}{T_{(i,t-1)}}}$，因此$\mu_i$的上界为$\bar{x}_{i,T_{(i,t-1)}} + \sqrt{\frac{2\ln t}{T_{(i,t-1)}}}$。当然，要选择一个随着$t$取值增大而收敛到0的函数，未必一定要选$t^{-4}$，如果换成关于$t$的其它幂函数，结果将会和当前$\delta$取值相差一个常数系数，因此奖励期望的上界也可以写成$\bar{x}_{i,T_{(i,t-1)}} + C\sqrt{\frac{2\ln t}{T_{(i,t-1)}}}$，其中$C$是一个预先指定的超参数，可以理解为用来调节探索和利用两者权重的因子。

蒙特卡洛树搜索：上限置信区间算法策略

UCB1算法的策略可以描述为，在第t次时选择使得上述不等式右侧足够小的动作a_{l_t}，其中l_t由如下式子计算得到：

$$l_t = \text{argmax}_i \, \bar{x}_{i,T_{(i,t-1)}} + C\sqrt{\frac{2\ln t}{T_{(i,t-1)}}}$$

文献证明了：UCB1算法在经过T次测试后，悔值函数的期望上界为$O(\frac{K\ln T}{\Delta})$，

其中$\Delta = \min_{\substack{i=1,\cdots,K; \\ \mu_i<\mu^*}} \mu^* - \mu_i$

蒙特卡洛树搜索

无论是一般搜索问题，还是对抗搜索问题，在问题特别复杂时，搜索树可能会变得十分巨大，以至于搜索算法很难在短时间内完全探索整棵搜索树。

为了解决这个问题，前面章节分别探讨了如何利用辅助信息来找到高效的结点扩展顺序以及介绍了可减少不必要扩展的结点数量的alpha-beta剪枝算法。

不难发现，对搜索算法进行优化以提高搜索效率基本上是在解决如下两个问题：优先扩展哪些结点以及放弃扩展哪些结点，综合来看也可以概括为如何高效地扩展搜索树。

如果将目标稍微降低，改为求解一个近似最优解，则上述问题可以看成是如下探索性问题：算法从根结点开始 ，每一步动作为选择（在非叶子结点）或扩展（在叶子结点）一个孩子结点。可以用执行该动作后所收获奖励来判断该动作优劣。奖励可以根据从当前结点出发到达目标路径的代价或游戏终局分数来定义。算法会倾向于扩展获得奖励较高的结点。

算法事先不知道每个结点将会得到怎样的代价（或终局分数）分布，只能通过采样式探索来得到计算奖励的样本。由于这个算法利用蒙特卡洛法通过采样来估计每个结点的价值，因此它被称为**蒙特卡洛树搜索**（Monte-Carlo Tree Search）算法。

蒙特卡洛树搜索

- **选择（selection）**：选择指算法从搜索树的根结点开始，向下递归选择子结点，直至到达叶子结点或者到达尚未被完全扩展的的结点L，如图(a)所示。这个向下递归选择过程可由UCB1算法来实现，在递归选择过程中记录下每个结点被选择次数和每个结点得到的奖励均值。
- **扩展（expansion）**：如果结点 L 不是一个终止结点（或对抗搜索的终局结点），则随机扩展它的一个未被扩展过的后继结点M，如图 (b)所示。
- **模拟（simulation）**：从结点M出发，模拟扩展搜索树，直到找到一个终止结点，如图 (c)所示。模拟过程使用的策略和采用UCB1算法实现的选择过程并不相同，前者通常会使用比较简单的策略，例如使用随机策略。
- **反向传播（Back Propagation）**：用模拟所得结果（终止结点的代价或游戏终局分数）回溯更新模拟路径中M以上（含M）结点的奖励均值和被访问次数，如图 (d)所示。

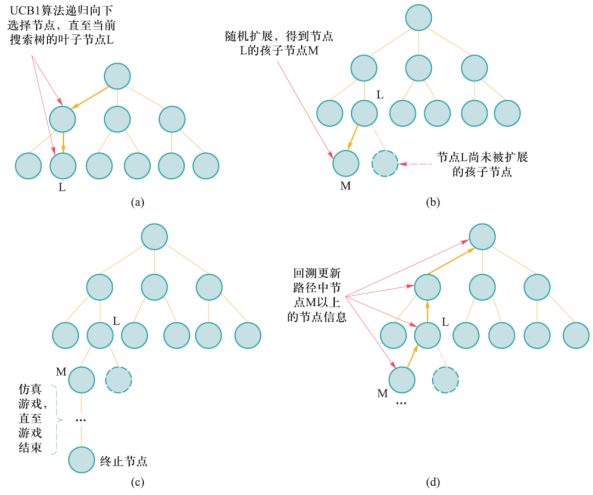
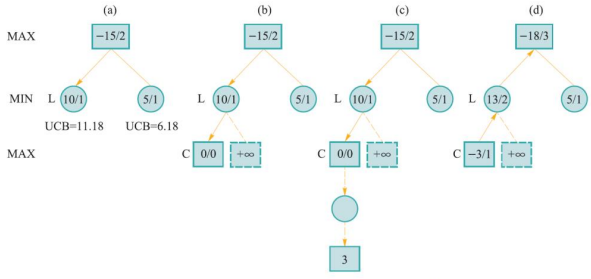


图 3.20　蒙特卡洛树搜索的四个步骤

蒙特卡洛树搜索

以Minimax算法为基础的蒙特卡洛树搜索算法也被称为上限置信区间树搜索（Upper Confidence Bounds for Trees, UCT）算法。图3.21展示了蒙特卡洛树搜索的一次迭代中四个完整步骤，其中结点中数字记录了经过该结点所得收益分数之和（左侧数字）和访问该结点的次数（右侧数字）。

在图 (a)中，搜索树中已经有三个结点，注意此时根结点的分数之和为负数，这是用蒙特卡洛树搜索求解对抗搜索问题时的特殊措施，原因我们将在讨论反向传播步骤时提到。此时分别计算第二层结点的UCB值：左侧结点为$\frac{10}{1}+\sqrt{\frac{2\ln 2}{1}}=11.18$，右侧结点为$\frac{5}{1}+\sqrt{\frac{2\ln 2}{1}}=6.18$，因此算法选择第二层左侧的结点L，由于该结点有尚未扩展的子结点，因此选择阶段结束。



（注：结点中的数字代表总收益分数 / 被访问次数）

图 3.21 蒙特卡洛树搜索算法

蒙特卡洛树搜索算法。结点中数字代表总收益分数/被访问次数

蒙特卡洛树搜索

在图 (b)中，算法随机扩展了L的子结点C，将其总分数和被访问次数均初始化为0。注意，为了清晰地展示算法选择扩展的结点，图(b)画出了L的其他未被扩展的子结点，并标记其UCB值为正无穷大，以表示算法下次访问到L时必然扩展这些未被扩展的结点。图(c)中采用随机策略模拟游戏直至完成游戏。当游戏完成时，终局得分为3。

在图 (d)中C结点的总分被更新为-3，被访问次数被更新为1；L结点的总分被更新为13，被访问次数被更新为2；根结点的总分被更新为-18，被访问次数被更新为3。在更新时，会将MIN层结点现有总分加上终局得分分数，MAX层结点现有总分减去终局得分分数。这是因为在对抗搜索中，玩家MIN总是期望最小化终局得分，因此在MIN层选择其子结点时，其目标并非选取奖励最大化的子结点，而是选择奖励最小化的结点，为了统一使用UCB1算法求解，算法将MIN层的子结点（即MAX层结点）的总分记为其相反数。

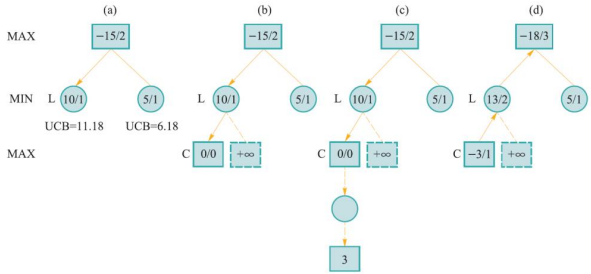


图 3.21 蒙特卡洛树搜索算法

（注：结点中的数字代表总收益分数 / 被访问次数）

蒙特卡洛树搜索算法。结点中数字代表总收益分数/被访问次数

蒙特卡洛树搜索

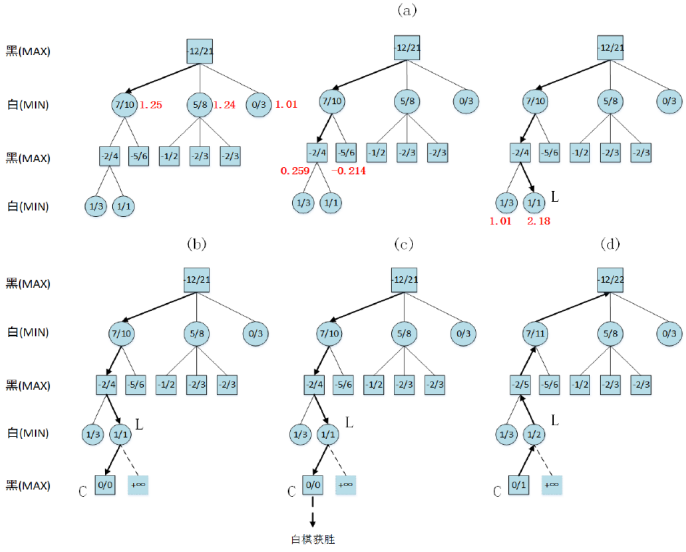


图3.22 一个更复杂的蒙特卡洛树搜索的例子
（UCB值在对应结点边用红色字体标出）

蒙特卡洛树搜索

算法 3.4 基于最小最大搜索的蒙特卡洛树搜索算法

函数： UCTSearch
输入： 当前状态 s_0
输出： 玩家 MAX 行动下，当前最优动作 a^*

```
1 v0 ← create_node(s0)
2 while 未达到最大迭代次数 do
3 |  vl ← SelectPolicy(v0)
4 |  sl ← SimulatePolicy(vl.state)
5 |  BackPropagate(vl, sl)
6 end
7 a* ← UCB1(v0, 0)
```

函数： SelectPolicy
输入： 选择的起始结点 v_0
输出： 选择步骤的结束结点 v

```
1 v ← v0
2 while not terminal_test(v.state) do
3 |  if v 存在未被扩展的子结点 then
4 |  |  return Expand(v)
5 |  else
6 |  |  v ← UCB1(v, Cp)
7 |  end
8 end
```

函数： SimulatePolicy
输入： 状态 s_0
输出： 模拟的终止状态 s

```
1 s ← s0
2 while not terminal_test(s) do
3 |  a ← 从 actions(s) 中随机采样
4 |  s ← result(s, a)
5 end
```

函数： BackPropagate
输入： 反向传播更新的起始结点 v，终局状态 s_t

```
1 while v is not null do
2 |  v.N ← v.N + 1
3 |  v.Q ← v.Q + utility(st, player(v.state))
4 |  v ← v.parent
5 end
```

函数： Expand
输入： 结点 v
输出： 未被扩展的后继结点 v'

```
1 a ← actions(v.state) 中随机的未探索动作
2 v' ← create_node(result(v, state, a))
3 v'.N ← 0
4 v'.Q ← 0
5 v'.parent ← v
6 v'.children ← ∅
7 v.children ← v.children ∪ {v'}
```

函数： UCB1
输入： 结点 v，超参数 c
输出： 置信上限最大的动作 a^*

$$a^* \leftarrow \operatorname{argmax}_{v' \in v.\text{children}} \frac{v'.Q}{v'.N} + c\sqrt{\frac{2\ln v.N}{v'.N}}$$

延伸阅读

突破组合爆炸之难与陷入地平线问题之困

香农之数：在象棋比赛中，深蓝需要判断某一时刻棋局落子对整个棋局胜负会带来怎样的影响。由于可选棋局众多，尽管深蓝平均每秒能够对1亿个棋局进行判断评估，还是无法在规定时间内计算得到当前棋局对胜负的潜在影响。1950年，香农（Claud Shannon）发表了一篇有关国际象棋编程的论文。在这篇论文中，香农估算国际象棋比赛中落子选择从第一次移动时的20种会增加到第二次移动时的400种，在第六次移动时可能的落子选择达到1.19亿种。香农甚至估算认为国际象棋的落子总数为10的120次方种，远远超出了10的82次方这一宇宙原子总数，这就是国际象棋中“组合爆炸”难题。

剪枝--眼不见心不烦：为了从海量可能落子中选择一种合适落子，以克服组合爆炸挑战，深蓝采用了由1971年图灵奖获得者约翰 麦卡锡 (John McCarthy)发明的“阿尔法-贝塔（Alpha-Beta）”剪枝搜索算法。

上帝之落子：2016年3月，AlphaGo在首尔以4:1比分战胜了李世石。李世石在第四局78步给出了惊人落子，扭转了乾坤。AlphaGo无法应对李世石这一步神来落子是搜索算法固有存在地平线问题（horizon problem）所致，即搜索算法无法看到远方地平线以下的景象。这可理解为“计算机暴力搜索计算”与“人类直觉顿悟”之间的差异，突破地平线之困依靠搜索之能还相距甚远。

本章小结

- 根据所设定问题，直接从搜索空间中寻找符合条件的答案。如果搜索空间较大，则需要采取剪枝和采样等手段，前者与搜索树结点出现的位置次序有关、后者则需要对可能结点进行大量仿真模拟。
- **搜索是人工智能智能能力体现的一种手段，匹配式搜索与数据采样式学习相互结合，使得搜索超越了传统匹配计算模式。**

谢谢观看